

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0611

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: *Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)*

Assessment Tool Weightage: 40%

QUESTIONS: 4

Total Marks:40

Annexure A

Constraint Based Problem Solving

Code of Conduct:

*I DHILIBAN M /192421166 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

DHILIBAN M

Annexure A

Short Answer Test

- 1, You are given a large dataset of patient health records stored in a CSV file, containing missing values, inconsistent formats, and outliers. Using NumPy and Pandas, design a constraint-based data cleaning pipeline that ensures: (i) no missing values remain, (ii) all numerical values fall within medically valid ranges, and (iii) categorical fields are standardized. Explain how you will use indexing, filtering, and aggregation functions to enforce these constraints while preserving maximum data integrity.

3. Two large Pandas DataFrames containing customer transaction and demographic data must be combined. However, constraints include: (i) no duplication of customer IDs, (ii) consistent indexing after merging, and (iii) preservation of all valid records. Describe how you will perform the combining operation, validate constraints using filtering and grouping, and ensure correctness through sorting and indexing techniques. Include steps to debug mismatched or missing entries.

4. You are tasked with computing statistical measures (mean, median, variance) on a dataset using NumPy and Pandas, but with constraints on both computational efficiency and numerical accuracy. Explain how you will implement aggregation and statistical functions while avoiding precision loss and redundant computations. Discuss how filtering and indexing can be used to exclude irrelevant data points, and how you would debug incorrect statistical outputs.

5. Design a complete data analysis pipeline that reads multiple data files, processes them using Pandas DataFrames, and generates visualizations. Constraints include: (i) efficient file I/O operations, (ii) correct grouping and sorting of data, (iii) filtering based on user-defined conditions, and (iv) meaningful plotting outputs. Explain how each stage (loading, processing, aggregation, and plotting) will be implemented and optimized, along with debugging steps for handling corrupted files or inconsistent data formats.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Q1. Constraint-Based Data Cleaning Pipeline for Patient Health Records

Conceptual Overview

Cleaning a large, messy patient CSV requires a **pipeline** with distinct, verifiable stages, each enforcing one constraint at a time so integrity can be checked incrementally rather than all at once. The three constraints map directly to three pipeline stages: **(i) missing-value handling**, **(ii) range/validity enforcement**, and **(iii) categorical standardization**.

Pipeline Design

Stage 0 — Load & Profile

```
python
import pandas as pd
import numpy as np

df = pd.read_csv("patients.csv")
print(df.info())          # dtypes, non-null counts
print(df.isna().sum())    # missing values per column
print(df.describe())      # spot outliers via min/max/percentiles
```

Profiling first prevents blind cleaning — we see *which* columns need which treatment before applying constraints.

Stage 1 — Handle Missing Values (no NaNs remain)

- Use `df.isna()` / `df.isnull()` for indexing to locate gaps.
- For numeric columns: impute with **median** (robust to outliers) grouped by a relevant category, e.g. age-band or gender, rather than a single global mean:

```
python
df["blood_pressure"] = df.groupby("age_group")["blood_pressure"] \
    .transform(lambda x: x.fillna(x.median()))
```

- For categorical columns: impute with the **mode**, or an explicit "Unknown" label if the missingness itself is informative:

```
python
df["gender"] = df["gender"].fillna(df["gender"].mode()[0])
```

- Rows where **critical identifiers** (e.g., `patient_id`) are missing are dropped, since they cannot be safely imputed:

```
python
df = df.dropna(subset=["patient_id"])
```

- Final constraint check: `assert df.isna().sum().sum() == 0`

Stage 2 — Enforce Medically Valid Ranges Define a dictionary of valid ranges (domain knowledge) and use **boolean-mask filtering** with NumPy for vectorized enforcement:

```
python
valid_ranges = {
    "age": (0, 120),
    "heart_rate": (30, 220),
    "blood_pressure_systolic": (60, 250),
    "temperature_c": (30, 43)
}

for col, (low, high) in valid_ranges.items():
    mask_invalid = ~df[col].between(low, high)
    df.loc[mask_invalid, col] = np.nan      # flag out-of-range as missing
    df[col] = df[col].fillna(df[col].median()) # re-impute instead of dropping, to preserve
records
```

Using `np.where/Series.clip()` is an alternative for capping outliers instead of discarding them:

```
python
df["age"] = df["age"].clip(lower=0, upper=120)
```

Clipping vs. re-imputation is chosen depending on whether the value is a plausible (but extreme) measurement vs. a clear data-entry error.

Stage 3 — Standardize Categorical Fields

```
python
df["gender"] = df["gender"].str.strip().str.lower()
df["gender"] = df["gender"].replace({
    "m": "male", "f": "female", "male ": "male", "fem": "female"
})
df["gender"] = df["gender"].astype("category")
```

`str.strip()`, `str.lower()`, and mapping dictionaries standardize inconsistent free-text entries (e.g., "M", "male", " Male") into a single canonical category.

Preserving Data Integrity

- Prefer **imputation/clipping over row deletion** wherever possible, since dropping rows loses entire patient records; deletion is reserved only for unrecoverable identifier fields.
- Use `.copy()` when subsetting to avoid `SettingWithCopyWarning` and unintended mutation of the original `DataFrame`.
- Track integrity with aggregation checks after each stage: `df.agg(['min','max','count'])` to confirm no invalid values or missing counts remain.
- Use indexing (`df.loc[mask, col]`) rather than iterating row-by-row, which is both faster and less error-prone.

Summary

The pipeline uses **profiling** → **missing-value imputation** → **range constraint enforcement (via boolean masking/clipping)** → **categorical standardization (via string cleaning + mapping)**, validated at each stage using indexing, filtering, and aggregation, to guarantee no missing values, medically valid ranges, and consistent categories while retaining the maximum number of usable records.

Q3. Combining Two Large DataFrames (Transactions + Demographics) Under Constraints

Conceptual Overview

Combining transaction data (many rows per customer) with demographic data (one row per customer) is a **many-to-one merge**. The constraints — no duplicate customer IDs, consistent indexing, and preservation of all valid records — dictate the merge type, the pre-merge validation, and the post-merge checks.

Step 1 — Pre-Merge Validation

```
python
# Check for duplicate customer IDs in the demographics table (should be unique)
dup_ids = demo_df["customer_id"].duplicated().sum()
assert dup_ids == 0, "Demographics table has duplicate customer IDs"

# If duplicates exist, resolve before merging
demo_df = demo_df.drop_duplicates(subset="customer_id", keep="last")
```

Step 2 — Choose the Correct Merge/Join

Since we want to **preserve all valid transaction records** (even if demographic info is missing for some), a **left join** on the transaction table is appropriate:

```
python
merged = trans_df.merge(
    demo_df,
    on="customer_id",
    how="left",
    validate="many_to_one", # enforces the no-duplication constraint explicitly
    indicator=True         # adds a _merge column for debugging
)
```

The `validate="many_to_one"` parameter causes Pandas to **raise an error automatically** if the demographic table contains duplicate `customer_ids` — this directly enforces constraint (i) at merge time rather than relying only on a manual check.

Step 3 — Consistent Indexing After Merging

```
python
```

```
merged = merged.reset_index(drop=True) # avoid old, fragmented indices
merged = merged.sort_values("customer_id").reset_index(drop=True)
```

`reset_index(drop=True)` ensures a clean, sequential index after the merge (merges often carry over misaligned indices from both source frames).

Step 4 — Validate Constraints via Filtering & Grouping

```
python
# (i) No duplication check post-merge
assert merged["customer_id"].duplicated(subset=["customer_id", "transaction_id"]).sum() == 0

# (iii) Preservation of all valid records
assert len(merged) == len(trans_df), "Row count changed unexpectedly after merge"

# Grouping to sanity-check aggregate consistency
check = merged.groupby("customer_id")["transaction_amount"].sum()
```

Grouping after the merge (`groupby("customer_id")`) is a strong validation tool: if totals per customer don't match pre-merge aggregates from `trans_df`, the merge introduced row duplication or loss.

Step 5 — Debugging Mismatched or Missing Entries

```
python
unmatched = merged[merged["_merge"] == "left_only"]
print(f'{len(unmatched)} transactions have no matching demographic record')

# Inspect why: possible ID format mismatch (e.g., string vs int, leading zeros)
print(trans_df["customer_id"].dtype, demo_df["customer_id"].dtype)
trans_df["customer_id"] = trans_df["customer_id"].astype(str).str.strip()
demo_df["customer_id"] = demo_df["customer_id"].astype(str).str.strip()
```

Common mismatch causes: differing dtypes (int vs str), whitespace, or leading zeros lost during CSV parsing — all fixed via type casting and `.str.strip()` before re-merging.

Summary

The combining strategy is: **validate uniqueness** → **left-merge with `validate="many_to_one"`** → **reset/sort index** → **verify row counts and grouped aggregates** → **debug unmatched IDs using the `_merge` indicator and dtype inspection**. This guarantees no duplicate customer IDs, a clean consistent index, and zero loss of valid transaction records.

Q4. Computing Statistical Measures Efficiently and Accurately (Mean, Median, Variance)

Conceptual Overview

The constraints here are twofold: **computational efficiency** (avoid redundant passes over data) and **numerical accuracy** (avoid precision loss, especially with floating-point summation on large arrays).

Efficient, Accurate Implementation

Use vectorized NumPy/Pandas built-ins rather than manual loops:

```
python
import numpy as np
import pandas as pd

# Single pass, vectorized (fast — implemented in C internally)
mean_val = df["cholesterol"].mean()
median_val = df["cholesterol"].median()
var_val = df["cholesterol"].var(ddof=1)  # ddof=1 -> sample variance (unbiased)
```

Vectorized operations avoid Python-level loops (which are slow and reduce numerical precision due to repeated intermediate rounding in naive accumulation).

Avoiding precision loss:

- Use `np.float64` (double precision) rather than `float32` for accumulation-heavy statistics, since summing many `float32` values compounds rounding error.
- For very large datasets, NumPy's `np.sum` / Pandas' `.sum()` use **pairwise summation** internally, which is more numerically stable than naive sequential summation — prefer these over manual accumulation loops.
- For variance specifically, avoid the naive formula $E[X^2] - (E[X])^2$ (which can suffer from **catastrophic cancellation** when values are large but variance is small). Instead rely on NumPy/Pandas' internal implementation, which uses a **two-pass, mean-centered algorithm**:

```
python
# Internally equivalent to a numerically stable two-pass method:
mean = arr.mean()
var = ((arr - mean) ** 2).mean()  # more stable than E[X^2]-E[X]^2 for large-magnitude data
```

Avoiding redundant computation:

```
python
# Bad: recomputes mean multiple times inside a loop-like structure
# Good: compute once, reuse
stats = df["cholesterol"].agg(["mean", "median", "var", "std"])  # single aggregation call
```


Using a single `.agg()` call computes all statistics together, avoiding multiple separate passes over the same column.

Using Filtering & Indexing to Exclude Irrelevant Data

```
python
# Exclude irrelevant/invalid rows before computing statistics
valid = df[(df["cholesterol"] > 50) & (df["cholesterol"] < 500) & df["cholesterol"].notna()]
mean_val = valid["cholesterol"].mean()
```

Boolean filtering ensures statistics reflect only medically plausible, non-null values — including outliers or NaNs would silently bias the mean/variance (NaNs are auto-excluded by `.mean()/var()` by default via `skipna=True`, but explicit filtering is safer for outliers).

Debugging Incorrect Statistical Outputs

1. **Check for NaNs/Infs first:** `df["col"].isna().sum(), np.isinf(df["col"]).sum()` — a single inf can silently break mean/variance.
2. **Check dtype:** ensure the column is numeric (`df["col"].dtype`); a column read as object (mixed strings/numbers) will produce wrong or error-prone aggregates.
3. **Compare against a manual small-sample calculation:** verify `df["col"].head(5).mean()` against manual arithmetic to catch logic errors.
4. **Check for duplicate rows** inflating/deflating aggregate statistics: `df.duplicated().sum()`.
5. **Verify ddof setting** — using `ddof=0` (population variance) vs `ddof=1` (sample variance, Pandas' default) is a very common source of "incorrect" variance values.

Summary

Efficiency and accuracy are achieved by using **vectorized built-in aggregation functions** (single-pass, C-optimized, numerically stable), **filtering out invalid/irrelevant rows before aggregation**, and **combining multiple statistics into one `.agg()` call**. Debugging focuses on checking for NaN/Inf contamination, dtype issues, duplicate rows, and ddof misconfiguration.

Q5. Complete Data Analysis Pipeline: Load → Process → Aggregate → Plot

Pipeline Architecture

[Multiple Files] -> [Efficient I/O] -> [Cleaning/Processing] -> [Grouping/Sorting/Filtering] -> [Aggregation] -> [Visualization]

Stage 1 — Efficient File I/O

```
python
import pandas as pd
import glob

files = glob.glob("data/sales_*.csv")
```

Efficient loading: specify dtypes upfront, read only needed columns

```
dfs = []
```

```
for f in files:
```

```
    try:
```

```
        chunk = pd.read_csv(
```

```
            f,
```

```
            usecols=["date", "store_id", "product", "sales", "region"],
```

```
            dtype={"store_id": "int32", "sales": "float32"},
```

```
            parse_dates=["date"]
```

```
        )
```

```
        dfs.append(chunk)
```

```
    except (pd.errors.ParserError, UnicodeDecodeError) as e:
```

```
        print(f'Corrupted file skipped: {f} -> {e}')
```

```
df = pd.concat(dfs, ignore_index=True)
```

- usecols and explicit dtype reduce memory footprint and parsing time significantly for large files.
- For very large files that don't fit in memory, `pd.read_csv(f, chunksize=100000)` processes data in batches.
- Wrapping each file read in try/except handles **corrupted files** gracefully without crashing the whole pipeline.

Stage 2 — Processing (Cleaning + Standardization)

```
python
```

```
df = df.drop_duplicates()
```

```
df["region"] = df["region"].str.strip().str.title()
```

```
df = df.dropna(subset=["sales"])
```

Stage 3 — Grouping, Sorting, Filtering

```
python
```

```
# Filtering based on user-defined condition
```

```
min_sales = 100
```

```
filtered = df[df["sales"] >= min_sales]
```

```
# Grouping and aggregation
```

```
summary = (
```

```
    filtered.groupby(["region", "product"])
```

```
        .agg(total_sales=("sales", "sum"), avg_sales=("sales", "mean"), n=("sales", "count"))
```

```
        .reset_index()
```

```
)
```

```
# Sorting
```

```
summary = summary.sort_values("total_sales", ascending=False)
```

`groupby().agg()` with named aggregation is preferred over multiple separate `groupby` calls, since it computes all required statistics in a single grouped pass — more efficient than repeated grouping.

Stage 4 — Visualization

python

```
import matplotlib.pyplot as plt
```

```
top10 = summary.head(10)
plt.figure(figsize=(10, 6))
plt.bar(top10["region"] + " - " + top10["product"], top10["total_sales"])
plt.xticks(rotation=45, ha="right")
plt.ylabel("Total Sales")
plt.title("Top 10 Region-Product Combinations by Total Sales")
plt.tight_layout()
plt.savefig("top_sales.png")
plt.show()
```

Plots are made **meaningful** by limiting to top-N results (avoiding unreadable overcrowded charts), labeling axes, and using `tight_layout()` to prevent clipped labels.

Optimization Considerations

- Use `pd.concat(dfs, ignore_index=True)` once at the end rather than repeatedly appending inside the loop (repeated appends are $O(n^2)$).
- Downcast numeric dtypes (`float32/int32`) to reduce memory for large files.
- Use `df.groupby(..., observed=True)` when grouping on categorical columns to avoid unnecessary combinations.

Debugging Corrupted Files / Inconsistent Formats

1. **Wrap I/O in try/except** (shown above) to isolate and log bad files without halting the pipeline.
2. **Check column consistency across files** before concatenation:

python

```
for f in files:
```

```
    cols = pd.read_csv(f, nrows=0).columns.tolist()
```

```
    print(f, cols) # detect files with missing/extra/renamed columns
```

3. **Validate schema after concat:** `df.dtypes` to catch columns that became object due to mixed types across files (e.g., one file had "N/A" strings in a numeric column).
4. **Row-count reconciliation:** compare `sum(len(pd.read_csv(f)) for f in files)` against `len(df)` to detect silent row loss.
5. **Use errors="coerce"** with `pd.to_numeric/pd.to_datetime` to convert malformed values to NaN instead of crashing, then audit how many were coerced.

Summary

The pipeline is built as **fault-tolerant I/O (chunked/selective reading with error handling) → cleaning/standardization → filter/group/sort using vectorized Pandas operations with named aggregation → optimized, labeled visualization**, with debugging built in at each stage via schema checks, row-count reconciliation, and explicit coercion of malformed values.